Python Generator 各原语参数说明

0x02 Axon （ΣX+Bias\b）
0x03 Axon（X1•X2+Bias\b）
0x04 Axon MLP
0x06 Soma Move Merge
0x26 Soma Move Split
0x07 Soma LUT
0x08 Soma LIF+
0x41 Axon CNN0
0x43 Axon（A•X+Bias\b）
0x81 Axon CNN1
0x83 Axon（aX+Bias\b）
0xX5 Soma Pooling
0x09 Router

机密

0x02 Axon（ΣX+Bias\b）

参数	说明	范围
avg_pooling_en	pooling 模式/向量相加模式	True/false
pad_on	是否 pad	True/false
InA_type	输入 X 类型	[0]int32[1]int8[2]uint8 [3]Tenary
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置[2/3]向量偏置 Bias
cin	输入向量 channel 个数	根据不同数据类型配置 32B 的整数倍
Bias_length	偏置向量长度	32n(即保证 32B 整数倍)
Input_fm_Px	输入 FM 宽度	非硬件底层实际原语参数
Input_fm_Py	输入 FM 高度	非硬件底层实际原语参数
pad_top	图像上方补零行数	0~7
pad_down	图像下方补零行数	0~7
pad_left	图像左面补零列数	0~7
pad_right	图像右面补零列数	0~7
pooling_Kx	Pooling 核宽度 （Array_num = Kx×Ky）	1~16
pooling_Ky	Pooling 核高度 （Array_num = Kx×Ky）	1~16
pooling_Sx	X 方向步长（仅池化模式）	1~7
pooling_Sy	Y 方向步长（仅池化模式）	1~7
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 V 地址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(向量数据 X)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	偏置向量的初始地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

注意：

1. 做平均池化且有 padding 时，原语参数 Addr_InA_base 会在用户配置的基础上做一个偏移。即最终原语参数的 Addr_InA_base = Addr_InA_base(用户配置) – Addr_start_offset，但输入数据的实际存储位置仍是用户配置的 Addr_InA_base。该过程有软件自动完成。
2. 做平均池化且没有 padding 时，即 pad_on = False 时，四个方向的 pad 参数都会置零
3. avg_pooling_en = False 时，就可以做普通的向量累加，其中累加向量的次数(X1+X2+X3+…+Xk)，k=Pooling_Kx×Pooling_Ky。每个输入向量 X 的深度是 cin，添加偏置向量 Bias 时，cin=Bias_length。每个输入向量 X 的数量由 Input_fm_Px×Input_fm_Py 决定，优先使用 Input_fm_Py（即 Input_fm_Px=1），fm 累加的时候再增加使用 Px。

4. 向量长度由 cin 确定：int32：8n，int8：32n，int2：128n
5. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好。

0x03 Axon（X1•X2+Bias\b）

参数	说明	范围
InA_type	InA 数据类型	[0]无效[1]int8[2]uint8 [3]无效
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置 [2/3]向量偏置 Bias
Bias_length	偏置向量 Bias 的向量长度	32 的整数倍、常数偏置时自动配零
tensor_en	是否进行张量运算	True/false
Px	张量运算的宽度	非硬件底层实际原语参数
Py	张量运算的高度	非硬件底层实际原语参数
stride_x	张量运算时的横向步长	非硬件底层实际原语参数
stride_y	张量运算时的纵向步长	非硬件底层实际原语参数
X_array_num	向量运算时的向量个数	非张量运算时，等于 L4_num + 1
cin	向量运算时的向量长度	向量偏置时，与 Bias_length 保持一致；常数偏置时，32 的整数倍
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 Vout 地址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(向量数据 X1)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_InB_base	输入数据 InA(向量数据 X2)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	偏置向量的初始地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

注意：

1. Tensor_en 有效时（此时 L5_num≠0），输入 X1 和 X2 都是张量（矩阵），宽度 Px，高度 Py，对两个张量做乘法。stride_x，stride_y 为步长，即可以实现跳着计算（为了验证 step 的多种可能）。这些参数都不是实际原语参数。转化后才可使用。
2. Tensor_en 无效时（此时 L5_num=0），执行正常向量相乘，X_array_num 是做相乘的向量个数，即 L4_num + 1
3. cin 是向量长度，当常数偏置时，cin 需配置成 32 的整数倍(32n)。当 Bias 偏置时，cin=Bias_length=32n。
4. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0x04 Axon MLP

参数	说明	范围
InA_type	InA 数据类型	[0]无效[1]int8[2]uint8[3]Tenary
InB_type	InB 数据类型	[0]无效[1]int8[2]uint8[3]Tenary
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置[2/3]向量偏置
cin	输入向量 X 的深度	InA 为 8bit 时，1~16*256 InA 为三值时，1~64*256
cout	输出向量的深度（Bias 偏置向量的深度）	InB 为 8bit 时，1~32*256 InB 为三值时，1~128*256
Bias_length	Bias 偏置向量深度	加载常数时，配置为 1；加载 Bias 时，cout=Bias_length
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 Vout 地址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(权重数据 X)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_InB_base	输入数据 InB(权重数据 W)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	偏置向量的初始地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

注意：

1. 对于 MLP 而言，权重矩阵的维度是 cin×cout，所以权重 8bit 时需配置为 32 的倍数，权重三值需要配置为 128 的倍数。Bias 的向量长度不用配置，等于 cout。
2. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0xX5 Soma Pooling

参数	说明	范围
PIC_Mode	Max/Min 比较	0：max 1：min
pad_on	Pad 使能	True/false
type_in	X 数据类型（DW）	0:int32，1:int8，2:无效 3:Ternary
type_out	Y 数据类型	0:int32，1:int8，2:无效 3:Ternary
cin	输入 channel 数	Type_in=0 时:1~1024 Type_in=1 时:1~4096 Type_in=3 时:1~16384
cout	输出 channel 数	Type_out=0 时:1~1024 Type_ out =1 时:1~4096 Type_ out =3 时:1~16384
Input_fm_Px	输入 FM 宽度	满足不等式*
Input_fm_Py	输入 FM 高度	满足不等式*
pad_top	图像上方补零行数	0~7
pad_down	图像下方补零行数	0~7
pad_left	图像左面补零列数	0~7
pad_right	图像右面补零列数	0~7
pooling_Kx	Pooling 核宽度	1~16
pooling_Ky	Pooling 核高度	1~16
pooling_Sx	X 方向步长	1~7
pooling_Sy	Y 方向步长	1~7
CMP_C	一组比较器池化层比较器初始值(常数)	int32
in_cut_start	输入 32b 输出 8b/2b 时的截取起始位	0~15:从最低位隔位截取
Reset_Addr_in	载入原语后复位 X 地址寄存器控制位	[0]不复位 [1]复位
Reset_Addr_out	载入原语后清除 Y 地址寄存器控制位	[0]不复位 [1]复位
Row_ck_on	行流水处理标志位	[0]不流水 [1]流水
Addr_start_in	输入数据 X 初始地址	有符号数，按 4B 地址填写 0x0000~0x8fff
Addr_start_out	输出数据 Y 初始地址	按 4B 地址填写 0x0000~0x8fff
in_row_max	Axon-Soma 缓存区可以存储的行数	1~256
mem_sel	输出结果写入 mem2/mem3 标志位	[0]写入 mem2 [1]写入 mem3

*分别满足以下不等式：

$1 \leqslant ((Input_fm_Px + Pad_left + Pad_right) - Pooling_Kx) / Pooling_Sx + 1 \leqslant 256$

$1 \leqslant ((Input_fm_Py + Pad_top + Pad_down) - Pooling_Ky) / Pooling_Sy + 1 \leqslant 256$

注意：

1. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好
2. 有 padding 时，原语参数 Addr_start_in 会在用户配置的基础上做一个偏移。即最终原语参数的 Addr_start_in = Addr_start_in (用户配置) – Addr_start_offset，但输入数据的实际存储位置仍是用户配置的 Addr_start_in。该过程有软件自动完成。

3. Cin、cout 都是实际大小，不是按照 16B 分组的数值

机密

0x06 Soma Move Merge

参数	说明	范围
length_in	输入向量长度 （注意：是实际个数，不是 16B 的个数）	Type_in=0 时:1~1024 Type_in=1 时:1~4096 Type_in=3 时:1~16384
length_ciso	第二个输入向量长度 （注意：是实际个数，不是 16B 的个数）	Type_in=0 时:1~1024 Type_in=1 时:1~4096 Type_in=3 时:1~16384
num_in	输入向量个数	1~4096
num_ciso	第二个输入向量个数	1~4096
length_out	输出向量长度 （注意：是实际个数，不是 16B 的个数）	Type_out=0 时:1~1024 Type_ out =1 时:1~4096 Type_ out =3 时:1~16384
num_out	输出向量个数	1~4096
type_in	输入数据类型	0:int32，1:int8，2：保留， 3:Ternary
type_out	输出数据类型	0:int32，1:int8，2：保留， 3:Ternary
in_cut_start	输入 32b 输出 8b/2b 时的截取起始位	0~15:从最低位隔位截取
Reset_Addr_in	载入原语后复位输入向量地址寄存器控制位	[0]不复位 [1]复位
Reset_Addr_out	载入原语后复位输出向量地址寄存器控制位	[0]不复位 [1]复位
Reset_Addr_ciso	载入原语后复位向量 3 的地址寄存器控制位： 整合：作为第二个输入向量 拆分：作为第二个输出向量	[0]不复位 [1]复位
Row_ck_on	行流水处理标志位	[0]不流水 [1]流水
Addr_start_in	输入向量初始地址	按 4B 寻址 0x0000~0x8fff
Addr_start_ciso	向量 3 初始地址 整合：第二个输入向量起始地址 拆分：第二个输出向量起始地址	按 4B 寻址 0x0000~0x8fff
Addr_start_out	输出向量起始地址	按 4B 寻址 0x0000~0x8fff
in_row_max	Axon-Soma 缓存区可以存储的行数	1~256
mem_sel	输出结果写入 mem2/mem3 标志位	[0]写入 mem2 [1]写入 mem3
in_ciso_pipe_sel	选择 axon 的流水结果作为 06 原语的 in 向量还是 ciso 向量	0：作为 in 向量 1：作为 ciso 向量
real_length_in_en	需要 end 地址不等于 start+num_in 时设置为 true	True/False

real_num_in	in 向量在存储中的实际个数（为节约存储设置，与实际的搬运次数 num_in 不同）	1~num_in

注意：

1. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好
2. Length 有关参数都是实际大小，不是按照 16B 分组的数值

机密

0x26 Soma Move Split

参数	说明	范围
length_in	输入向量长度（注意：是实际个数，不是 16B 的个数）	Type_in=0 时:1~1024 Type_in=1 时:1~4096 Type_in=3 时:1~16384
length_ciso	向量 3 长度： 整合：第二个输入向量长度 拆分：第二个输出向量长度 （注意：是实际个数，不是 16B 的个数）	Type_in=0 时:1~1024 Type_in=1 时:1~4096 Type_in=3 时:1~16384
num_in	输入向量个数	1~4096
num_ciso	向量 3 个数： 整合：第二个输入向量个数 拆分：第二个输出向量个数	1~4096
length_out	输出向量长度 （注意：是实际个数，不是 16B 的个数）	Type_out=0 时:1~1024 Type_ out =1 时:1~4096 Type_ out =3 时:1~16384
num_out	输出向量个数	1~4096
type_in	输入数据类型	0:int32，1:int8，2：保留， 3:Ternary
type_out	输出数据类型	0:int32，1:int8，2：保留， 3:Ternary
in_cut_start	输入 32b 输出 8b/2b 时的截取起始位	0~15:从最低位隔位截取
Reset_Addr_in	载入原语后复位输入向量地址寄存器控制位	[0]不复位 [1]复位
Reset_Addr_out	载入原语后复位输出向量地址寄存器控制位	[0]不复位 [1]复位
Reset_Addr_ciso	载入原语后复位向量 3 的地址寄存器控制位： 整合：作为第二个输入向量 拆分：作为第二个输出向量	[0]不复位 [1]复位
Row_ck_on	行流水处理标志位	[0]不流水 [1]流水
Addr_start_in	输入向量初始地址	按 4B 寻址 0x0000~0x8fff
Addr_start_ciso	向量 3 初始地址 整合：第二个输入向量起始地址 拆分：第二个输出向量起始地址	按 4B 寻址 0x0000~0x8fff
Addr_start_out	输出向量起始地址	按 4B 寻址 0x0000~0x8fff
in_row_max	Axon-Soma 缓存区可以存储的行数	1~256
mem_sel	输出结果写入 mem2/mem3 标志位	[0]写入 mem2 [1]写入 mem3

out_ciso_sel	选择与 router 流水的向量是 out 向量还是 ciso 向量	0：out 向量与 router 交互 1：ciso 向量与 router 交互
set_ciso_addr_end	是否自定义 ciso 向量的结束地址	True/False
Addr_end_ciso	（仅当 set_ciso_addr_end=True 时有效）设置 ciso 向量的结束地址（需设置 16B 对齐的地址，如果不对齐，自动向下对齐到 16B）	0x0000~0x8fff
set_out_addr_end	是否自定义 out 向量的结束地址	True/False
Addr_end_out	（仅当 set_out_addr_end =True 时有效）设置 out 向量的结束地址（需设置 16B 对齐的地址，如果不对齐，自动向下对齐到 16B）	0x0000~0x8fff

注意：

1. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好
2. Length 有关参数都是实际大小，不是按照 16B 分组的数值

机密

0x07 Soma LUT

参数	说明	范围
X_type	X 数据类型	0:int32，1\2\3:int8
Y_type	Y 数据类型	0:int32，1\2\3:int8
group_num	神经元组数	1~4096
neuron_real_num	每组神经元个数	int32:1~1024 int8:1~4096
LUT_DW	LUT 地址有效位宽	0:4b，1:8b，2:12b，3:16b
X_cut_start	输入数据的截取起始位	0~15:从最低位隔位截取
in_row_max	Axon-Soma 缓存区可以存储的行数	1~256
Reset_Addr_X	载入原语后复位 X 地址寄存器控制位:	[0]不复位 [1]复位
Reset_Addr_Y	载入原语后清除 Y 地址寄存器控制位:	[0]不复位 [1]复位
Row_ck_on	行流水处理标志位	[0]不流水 [1]流水
Addr_X_start	输入数据 X 初始地址	按 4B 寻址 0x0000~0x8fff
Addr_LUT_start	LUT 存储初始地址	按 4B 寻址 0x0000~0x8fff
Addr_Start_out	输出数据 Y 初始地址	按 4B 寻址 0x0000~0x8fff

注意:

1. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0x08 Soma LIF+

参数	说明	范围
neu_num	每组神经元个数	1~16×256,16B 的整数倍
group_num	神经元组数	1~4096
Seed	随机数种子	随机生成 int32
VM_const_en	随机掩膜测试开关	[0] 非随机掩膜 [1] 随机掩膜测试
VM_const	初始化不同神经元的掩膜为相同值的赋值	int28
VM_len	读取/写回 VM 长度（4B 个数）	随机掩膜测试时，为 neu_num*group_num，其余情况暂设为 4
Vtheta_const_en	自适应阈值测试开关	[0] 无自适应 [1] 常数自适应/指数自适应
Vtheta_const	无自适应阈值的常数值	int28，目前设为 0 即可
Vtheta_len	读取/写回 Vtheta 长度（4B 个数）	无自适应时，值为 4; 自适应阈值测试时，为 neu_num*group_num
Vth0	阈值的常值部分	随机生成，int28
Vth_adpt_en	指数自适应阈值使能标志	[0]不使能 [1]使能
Vth_alpha	自适应阈值衰减系数，除以 256 使用	uint8，无指数自适应阈值时设为 0
Vth_beta	自适应阈值衰减常数	int28，无指数自适应阈值时设为 0
Vth_Incre	自适应阈值增量	int28，无指数自适应阈值时设为 0
Vleaky_adpt_en	指数自适应膜电位泄露使能标志	[0]不使能 [1]使能
Vleaky_alpha	自适应膜电位泄露衰减系数，除以 256 使用	uint8，无指数自适应泄露时设为 0
Vleaky_beta	自适应膜电位泄露衰减常数	int28，无自适应泄露时设为 0
VR	膜电位复位值	int28
VL	膜电位下限值	int28
dV	阈值衰减固定值	int28
Vinit	神经元膜电位初始值	int28
Ref_len	神经元不应期长度	uint4
ref_cnt_const_en	不应期功能测试开关	[0] 不开启 [1] 开启
ref_cnt_const	初始化不同神经元为相同不应期（循环赋值）	uint4，不进行不应期测试时，设为 0；否则通常设为 1
Tw_en	膜电位按时间窗清零标志位	[0]不清零 [1]清零
Tw_len	输出时间窗长度 （膜电位复位周期）	无符号数，1-256

Tw_cnt	输出时间窗计数	uint8，不进行输出时间窗测试时设为 0
Rst_mode	复位模式选择标志位	0：复位为 VR 1：复位为膜电位减去阈值 2：复位为膜电位减去固定值 3：复位保持原值
fire_type	发放模式选择	0：复位衰减前的膜电位补成 32b 输出 1：复位衰减后的膜电位补成 32b 输出 2：复位衰减前的膜电位截取到 8b 输出 3：复位衰减后的膜电位截取到 8b 输出 4：spike 信号按 8b 输出 5：spike 信号按 2b 输出 6: 掩膜后的随机数按 32b 输出 7: 掩膜后的随机数按 8b 输出
in_cut_start	输入数据的截取起始位	0~12:从最低位隔位截取，注意不同发放模式的隔位截取范围不同
Row_ck_on	行流水启动标志	[0]不启动 [1]启动
in_row_max	Axon-Soma 缓存区可以存储的行数	单原语测试设为 0
reset_Addr_S	载入原语后清除 S 地址寄存器控制位	[0]不复位 [1]复位
reset_Addr_Uin	载入原语后复位 Uin 地址寄存器控制位:	[0]不复位 [1]复位
reset_Addr_V	载入原语后清除 V 地址寄存器控制位:	[0]不复位 [1]复位
reset_Addr_VM	载入原语后清除 VM 地址寄存器控制位:	[0]不复位 [1]复位
reset_Addr_Vtheta	载入原语后清除 Vtheta 地址寄存器控制位:	[0]不复位 [1]复位
Addr_Uin_start	数据 Uin 初始地址	按 4B 寻址 0x0000~0x8fff
Addr_S_Start	数据 S 初始地址	按 4B 寻址 0x0000~0x8fff
Addr_V_start	数据 V 起始地址	按 4B 寻址 0x0000~0x8fff
Addr_VM_start	数据 VM 起始地址	按 4B 寻址 0x0000~0x8fff
Addr_Vtheta_start	数据 Vtheta 起始地址	按 4B 寻址 0x0000~0x8fff
Addr_para	神经元静态参数存储地址	按 4B 寻址

		0x0000~0x8fff
mem_sel	输出结果写入 mem2/mem3 标志位	[0]写入 mem2 [1]写入 mem3

机密

0x41 Axon CNN0

参数	说明	范围
InA_type	InA(X)数据类型	[0]无效[01]int8[2]uint8 [3]Tenary
InB_type	InB(W)数据类型	[0]无效[01]int8[2]uint8 [3]Tenary
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置[2/3]向量偏置
Bias_length	偏置向量 Bias 的向量长度	32 的整数倍、常数偏置时自动配零
pad_on	是否启用 pad	True/false
Input_fm_Px	输入 FM 宽度	满足公式*
Input_fm_Py	输入 FM 高度	满足公式*
conv_Kx	卷积核长度	1~16
conv_Ky	卷积核高度	1~16
conv_Sx	卷积 x 方向步长	1~7
conv_Sy	卷积 y 方向步长	1~7
conv_Ex	卷积 x 方向膨胀系数	1~4
conv_Ey	卷积 y 方向膨胀系数	1~4
pad_top	图像上方补零行数	0~7
pad_down	图像下方补零行数	0~7
pad_left	图像左面补零列数	0~7
pad_right	图像右面补零列数	0~7
cin	输入 channel 数	InA 为 8bit 时，1~16*256 InA 为 3 值时，1~64*256
cout	输出 channel 数	1~32*256
Reset_Addr_A	载入原语后 InA 地址复位标识	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 InB 地址复位标识	[0]不复位 [1]复位
Addr_InA_base	InA 数据基地址	按 4B 寻址 0x0000~0x7fff
Addr_InB_base	InB 数据基地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出基地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	Bias 基地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动
axon_delay	配置这条 axon 原语是否用作扩充 phase 时钟	True/false （默认为 false）
L5_num	第 5 级循环（输出图像长度 Ox）计数最大值	0~255
L4_num	第 6 级循环（输出图像高度 Oy）计数最大值	0~255

* Input_fm_Px 和 Input_fm_Py 分别满足以下不等式：

$1 \le ((Input_fm_Px + Pad_left + Pad_right) - ((conv_Kx - 1) \times Conv_Ex + 1)) / Conv_Sx + 1 \le 256$

$1 \le ((Input_fm_Py + Pad_top + Pad_down) - ((conv_Ky - 1) \times Conv_Ey + 1))/ Conv_Sy + 1 \le 256$

注意：

（*）当 axon_delay 设置为 True 时，仅需要配置 L5_num、L4_num、Addr_InA_base、Addr_InB_base、Addr_V_base，A2S2_mode 即可，其他为默认值，Addr_V_end=Addr_V_base、Addr_InA_base= Addr_InA_end。

Axon 此时运行的实际时钟：6 + 37*（L4+1）*（L5+1）

1. 有 padding 时，注意把 Pad_on 设置为 true（四个方向都没有 padding 时设为 False）原语参数 Addr_InA_base 会在用户配置的基础上做一个偏移。即最终原语参数的 Addr_InA_base = Addr_InA_base(用户配置) – Addr_start_offset，但输入数据的实际存储位置仍是用户配置的 Addr_InA_base。该过程有软件自动完成。
2. InA、InB、Vout、Bias 存放时不能有地址冲突，软件目前没有设置提示，但是可以根据 para 文件的 Addr_end 判断

0x43 Axon（A•X+Bias\b）

参数	说明	范围
InA_type	InA 数据类型	[0]无效[1]int8[2]uint8 [3]三值
InB_type	InB 数据类型	[0]无效[1]int8[2]uint8 [3]无效
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置 [2/3]向量偏置 Bias
Bias_length	偏置向量 Bias 的向量长度	32 的整数倍、常数偏置时自动配零
tensor_en	是否进行张量运算	True/false，InA 三值不支持张量运算
Px	张量运算的宽度	非硬件底层实际原语参数
Py	张量运算的高度	非硬件底层实际原语参数
stride_x	张量运算时的横向步长	非硬件底层实际原语参数
stride_y	张量运算时的纵向步长	非硬件底层实际原语参数
X_array_num	向量运算时的向量个数	非张量运算时，等于 L4_num + 1
InB_length	向量运算时的向量长度(系数向量 A 的向量长度)	向量偏置时，与 Bias_length 保持一致；常数偏置时，32 的整数倍
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 Vout 地址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(向量数据 X1)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_InB_base	输入数据 InA(向量数据 X2)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	偏置向量的初始地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

注意：

1. Tensor_en 有效时（此时 L5_num≠0），输入 X 是张量（矩阵），宽度 Px，高度 Py，对两个张量做乘法。stride_x，stride_y 为步长，即可以实现跳着计算（为了验证 step 的多种可能）。这些参数都不是实际原语参数。转化后才可使用。
2. Tensor_en 无效时（此时 L5_num=0），执行正常向量相乘，X_array_num 是输入向量个数，即 L4_num + 1
3. InB_length 既是系数 A 向量的长度，也是输入向量 X 的长度。
4. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0x81 Axon CNN1

参数	说明	范围
Pad_on	Pad 使能	True/false
InA_type	InA(X)数据类型	[0]无效[1]int8[2uint8[3]无效
InB_type	InB(W)数据类型	[0]无效[1]int8[2uint8 [3]Ternary
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置 2/3]向量偏置
cin	输入 channel 数	InA 为 8bit 时，1~16*16 InA 为 3 值时，1~64*16
cout	输出 channel 数	1~32*256
Input_fm_Px	输入 FM 宽度	满足公式*
Input_fm_Py	输入 FM 高度	满足公式*
pad_up	图像上方补零行数	0~7，非实际原语参数
pad_down	图像下方补零行数	0~7，非实际原语参数
pad_left	图像左面补零列数	0~7，非实际原语参数
pad_right	图像右面补零列数	0~7，非实际原语参数
conv_Kx	卷积核高度	1~256
conv_Ky	卷积核宽度	1~16
conv_Sx	X 方向步长	1~7
conv_Sy	Y 方向步长	1~7，非实际原语参数
conv_Ex	X 方向膨胀系数	1~4
conv_Ey	Y 方向膨胀系数	1~4，非实际原语参数
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 InB 址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(权重数据 X)基地址	按 4B 寻址 0x0000~0x7fff
Addr_InB_base	输入数据 InB(权重数据 W)基地址	按 4B 寻址 0x0000~0x7fff
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	Bias 基地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

* Input_fm_Px 和 Input_fm_Py 分别满足以下不等式：

$1 \le ((Input_fm_Px + Pad_left + Pad_right) - ((conv_Kx - 1) \times Conv_Ex + 1)) / Conv_Sx + 1 \le 256$

$1 \le ((Input_fm_Py + Pad_top + Pad_down) - ((conv_Ky - 1) \times Conv_Ey + 1))/ Conv_Sy + 1 \le 256$

注意：

1. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0x83 Axon（aX+Bias\b）

参数	说明	范围
InA_type	InA 数据类型	[0]无效[1]int8[2]uint8 [3]无效
Load_Bias	Dendrite 赋初值标志位	[0/1]常数偏置 [2/3]向量偏置 Bias
Bias_length	偏置向量 Bias 的向量长度	32 的整数倍、常数偏置时自动配零
tensor_en	是否进行张量运算	True/false
Px	张量运算的宽度	非硬件底层实际原语参数
Py	张量运算的高度	非硬件底层实际原语参数
stride_x	张量运算时的横向步长	非硬件底层实际原语参数
stride_y	张量运算时的纵向步长	非硬件底层实际原语参数
X_array_num	向量运算时的向量个数	非张量运算时，等于 L4_num + 1
cin	向量运算时的向量长度	向量偏置时，与 Bias_length 保持一致；常数偏置时，32 的整数倍
Reset_Addr_A	载入原语后 InA 地址复位标志	[0]不复位 [1]复位
Reset_Addr_V	载入原语后 Vout 地址复位标志	[0]不复位 [1]复位
Addr_InA_base	输入数据 InA(向量数据 X)地址区间的初始地址	按 4B 寻址 0x0000~0x7fff
constant_a	常数系数 a	Int32 的值
Addr_V_base	输出 Vout 初始地址	按 4B 寻址 0x0000~0x7fff
Addr_Bias_base	偏置向量的初始地址	按 4B 寻址 0x0000~0x7fff
A2S2_mode	Axon 允许 Soma2 原语的启动方式	[0]Axon 计算完毕才允许启动 [1]Axon 未计算完也允许启动

注意：

1. Tensor_en 有效时（此时 L5_num≠0），输入 X 是张量（矩阵），宽度 Px，高度 Py，对两个张量做乘法。stride_x，stride_y 为步长，即可以实现跳着计算（为了验证 step 的多种可能）。这些参数都不是实际原语参数。转化后才可使用。
2. Tensor_en 无效时（此时 L5_num=0），执行正常向量相乘，X_array_num 是做相乘的向量个数，即 L4_num + 1
3. cin 是向量长度，当常数偏置时，cin 需配置成 32 的整数倍(32n)。当 Bias 偏置时，cin=Bias_length=32n。
4. 所有参数都是实际值，不用减 1。生成的原语参数会自动配置好

0x09 Router

参数	说明	范围
Rhead_mode	确定单个路由表发送数据数 Rhead_mode=1，每个包头占 8B Rhead_mode=0，每个包头占 4B	[0]单一数据路由表头 [1]路由表头发多个数据
T_mode	单包发送/多包发送标志位	[0]单包发送 [1]多包发送
Soma_in_en	是否和 Soma 进行数据交互的标志	[0]不和 Soma 进行数据交互 [1] 和 Soma 进行数据交互
CXY	当前 core 的中继/多播模式	[1]表示多播[2]表示中继 [0]/[3]表示无多播/中继
Send_en	本 core 是否发送数据标志位	[0]不发送数据 [1]发送数据
Receive_en	本 core 是否接收数据标志位	[0]不接收数据 [1]接收数据
Send_number	本 core 发送的路由包数（包含 EN=0 的数量）	0~4095 表示 1~4096
Dout_Mem_sel	待发送数据的 Memory 选择	0：Mem2 1：Mem3
Addr_Dout_base	Mem2 待发送数据的起始地址	0x000~0xfff，4B 寻址
Addr_Dout_length	Mem2 待发送数据的最大地址长度	无符号数，按 16B 计数 0~1023 表示 1~1024 个 16B
Addr_Rhead_base	Mem2 存储本地路由表的起始地址（配置以 mem2 为基址的偏移地址）	0x000~0xfff，4B 寻址
Addr_Rhead_length	Mem2 存储本地路由表包头长度	无符号数，按 16B 计数，0~255 表示 1~256 个 16B 数据
Addr_Din_base	接收数据在 Mem2 存储起始地址（配置以 mem2 为基址的偏移地址）	0x000~0xfff，4B 寻址
Addr_Din_length	接收数据在 Mem2 存储的长度	0~4095 表示 1~4096（单包时表示 1B 的长度（个数），多包时表示 8B 的长度（个数））
Receive_number	本 core 应接受结束标志总数，包括中继/多播包的结束标志	无符号数，0~4095 表示 1~4096
Relay_number	中继/多播包总数	0~4095 表示 1~4096
Nx	当前 core 的 x 方向多播步长	-64~63
Ny	当前 core 的 y 方向多播步长	-64~63
Send_PI_en	是否发送即时原语标志位	[0]不发送 [1]发送
Back_sign_en	是否需要返回应答	[0]不返回 [1]返回
Send_PI_num	应发送即时原语个数	无符号数，0~31 表示 1~32

Receive_sign_en	是否接受即时原语应答标志位	[0]不接收 [1]接受
Receive_sign_num	应接受即时原语应答个数	无符号数，0~31 表示 1~32
Send_PI_addr_base	X、Y、IN_PI 基地址	无符号数，地址按 16B 对齐， 表示范围：0~16KB 对应的地址
Q	即时原语应答包中多播标志位 Q	与包头中 Q 的定义一致

路由包头接口函数说明

函数参数分为两种形式：

当原语参数中 Rhead_mode=1 时，应使用如下形式：

addRHead(S=0, T=1, P=0, Q=0, X=0, Y=0, A=0, pack_per_Rhead=255, A_offset=0, Const=0, EN=1)

pack_per_Rhead:0 表示 1，范围 0~4095，表示这个包头发送 1~4096 个包

Const: 0 表示 1，范围 0~127，表示连续发送 1~128 个包

A_offset:0 表示 1，范围 0~4095，表示连续发送 Const+1 个包后 A 的偏移量，A=A+A_offset

当原语参数中 Rhead_mode=0 时，应使用如下形式：

addRHead(S=0, T=1, P=0, Q=0, X=0, Y=0, A=0)

addRHead 函数的调用顺序即为包头的存储顺序

IN_PI	X	Y	Q	P
11	8	8	1	4

R[31:21] = IN_PI，即时原语索引（PI_addr_offset+ A_valid+ S1_valid+ R_valid+ S2_valid）；

R[20:13] = dx[7:0]，目的簇的水平相对距离（int 8），范围[-128, 127]；

R[12:5] = dy[7:0]，目的簇的竖直相对距离（int 8），范围[-128, 127]；

R[4:4] = Q，多播与普通即时原语包标志位，P = 0 表示普通即时原语数据包，P = 1 表示多播即时原语数据包；

添加即时原语索引包接口函数说明

add_instant_pi(PI_addr_offset=0, A_valid=0, S1_valid=0, R_valid=1, S2_valid=0, X=1, Y=0, Q=0)

PI_addr_offset：0~63，（以 32B 为单位）

add_instant_pi 函数的调用顺序即为包头的存储顺序

core 间路由关系的表征：

当配置路由发送原语时，同时需要配置数据接收 core 的相关信息 send_destin_core_grp[]：

send_destin_core_grp[

{core_id: [((0,0),(0,1)) ,((0,0), (2,1))], data_num: 10, T_mode:0/1, Rhead_num:1, sync_en:0},

{core_id: [((0,0),(0,2))], data_num:10, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:2}]

数据要发送给多少个 core（非多播中继）就有几个字典。

其中，core_id 填写的是接收数据 core 的绝对坐标，每个接收数据 core 的绝对坐标都是二维元组，第一维(0,0)代表其所在的 chip 坐标，第二维代表其 chip 内坐标，均是先 x，后 y。

如果 core_id 这个列表中不止一个元素，如第一行例子，那么代表该发送 core 的这部分数据包要发给几个 core，list 中的 core（除最后一个）都是多播 core。core_id: [((0,0),(0,1)) ,((0,0),(2,1))]，就代表该 core 要发送数据给 core（0,1），core（0,1）再把数据多播给（2,1），（0,1）和（2,1）收到的都是同一块数据。

data_num：发送给对应 core 的数据包的个数

T_mode：单包/多包

Rhead_num: 使用表头的个数

sync_en: 跨组数据传输时是否需要 phase 同步（默认是 0，可以不写这个 key）

sync_phase_num：接收数据的 core 在执行第几个 phase 时收数据（从 0 开始，sync_en 为 1 时填写）

对应地，当配置路由为接收数据时，同时需要配置数据来源 core 的相关信息 recv_source_core_grp[]:

recv_source_core_grp [

{core_id: [((0,0),(0,1))], data_num: 10, T_mode:0/1, Rhead_num:1, sync_en:0},

{core_id: [((0,0),(0,3))], data_num:10, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:2}]

注意：sync_en 和 sync_phase_num(从 0 开始计数)，在发送和接收两端需要配置成一模一样

sync_phase_num 代表的是数据接收 core 在执行到第几个 phase 的时候接收收到这些数据包

(即时原语信息)发送(core_id 同样改为二维元组)

instant_prim_request[(core_id, index), (core_id, index)]

即时原语索引配置 PI_offset(7bit) + AS1RS2（4bit）

router_prim.add_instant_PI(PI_offset= , A_valid= ,S1_valid= ,R_valid= . S2_valid= , X= ,Y=, Q=)

(即时原语返回应答)Back_sign_en = 1 时,就是这个应答会返回给哪个 phase_grp 的所有 core_id

instant_request_back[((0,0),(0,0)), ((0,0),(0,1)), ((0,0),(0,2)), ((0,0),(0,3))]

路由配置实例：

(一)同时序簇(group)路由传输配置（含即时原语异步传输数据）

此时无需配置 sync_en 和 sync_phase_num 信息（或 sync_en=0）!

1. ((0,0),(0,0)) → ((0,0),(0,1))多播→ ((0,0),(0,2))中继→((0,0),(0,3))
   ((0,0),(0,0)) → ((0,0),(0,4))中继→ ((0,0),(0,5))
   ((0,0),(0,0)) → ((0,0),(0,6))多播→ ((0,0),(0,7))
   ((0,0),(0,0)) → ((0,0),(0,8))

   ((0,0),(1,0)) → ((0,0),(0,2))
   ((0,0),(1,0)) → ((0,0),(0,6))

   1)对于(0,0)需要配置 send_destin_core_grp 信息：

   注意：不体现中继 core 的信息，多播 core 需要体现

send_destin_core_grp[

{core_id: [((0,0),(0,1)) ,((0,0),(0,3))], data_num: 10, T_mode:0/1, Rhead_num:1},

{core_id: [((0,0),(0,5))], data_num:9, T_mode:0/1, Rhead_num:1},

{core_id: [((0,0),(0,6)), ((0,0),(0,7))], data_num:7, T_mode:0/1, Rhead_num:1},
{core_id: [((0,0),(0,8))], data_num:7, T_mode:0/1, Rhead_num:1}]

2)对于(0,4)中继 core，不需要配置 recv_source_core_grp 信息

3)对于(0,1)和(0,6)这样的多播 core，需要配置 recv_source_grp 信息：

((0,0),(0,1)):
recv_source_core_grp [{core_id: [((0,0),(0,0))], data_num: 10, T_mode:0/1, Rhead_num:1}]
((0,0),(0,6))，需要接收两个 core 的数据:
recv_source_core_grp [
{core_id: [((0,0),(0,0))], data_num: 7, T_mode:0/1, Rhead_num:1},
{core_id: [((0,0),(1,0))], data_num:10, T_mode:0/1, Rhead_num:1}]

4)对于(0,3)、(0,5)、(0,7)和(0,8),数据的最初来源都是(0,0)：

recv_source_core_grp [{core_id: [((0,0),(0,0))], data_num: ___, T_mode:0/1, Rhead_num:1}]

5)对于(0,2),一方面作为中继 core，一方面直接接收(1,0)数据，那么：

recv_source_core_grp [{core_id: [((0,0),(1,0))], data_num: 10, T_mode:0/1, Rhead_num:1}]

无需体现中继数据来源！

利用即时原语实现组间异步数据传输，配置方法同上。

(二)跨组数据同步传输的配置(需要配置 sync_en 和 sync_phase_num 参数)

当'sync_en'配置为 1 时，必须配置 sync_phase_num 参数!

Group0：((0,0),(0,0))
Group1：((0,0),(0,1))
Group2：((0,0),(0,2))

Group0 向 group1 发数据，group1 向 group2 发数据。其中 group1 需要在执行第 1 个 phase 就需要接收到 group0 第 1 个 phase 发来的数据。Group1 在第 2 个 phase 时发送数据给 group2, group2 在执行第 5 个 phase 就接收到 group1 数据。

关键配置如下：

Group0((0,0),(0,0))：
Phase1：
send_destin_core_grp[{core_id: [((0,0),(0,1))], data_num: 10, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:0}]

Group1((0,0),(0,1))：
Phase1:
recv_source_core_grp [{core_id: [((0,0),(0,0))], data_num: 10, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:0}] **后两个参数和 group0 的 send 配置一模一样
Phase2:
send_destin_core_grp [{core_id: [((0,0),(0,2))], data_num: 8, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:4}]

Group2((0,0),(0,2))：
Phase5:
recv_source_core_grp [{core_id: [((0,0),(0,1))], data_num: 8, T_mode:0/1, Rhead_num:1, sync_en:1, sync_phase_num:4}] **后两个参数和 group1 的 send 配置一模一样

机密